



Instituto Tecnológico de Costa Rica
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación, Ingeniería en Computación
II Semestre 2026

IC-6821 Diseño de Software

Tema 5: patrones de diseño
Investigación de patrones

Profesora:
Alicia Marcela Salazar Hernández

Estudiantes:
Andrés Campos Montero, 2025069367
Juan Vicente José Naranjo Molina, 2025076662

Fecha de entrega:
07/10/2026

1. Identidad

- **Nombre del patrón:** Builder.
- **Categoría:** Creación de objetos.
- **Fuente de estudio:** Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. pp. 97-106.
<https://www.javier8a.com/itc/bd1/articulo.pdf>

2. El problema y las fuerzas (CaféTrace)

Lo que se quería lograr: En CaféTrace, el Requisito 6 exige generar un reporte mensual para el ICAFE con un formato rígido de posiciones fijas definido en 1998. Al mismo tiempo, el sistema debe ser flexible para exportar la trazabilidad de los lotes de los últimos 5 años (R-10) en otros formatos legibles durante auditorías.

Lo que lo impedía: La rigidez del código tradicional. Si la lógica de extracción de datos se programa directamente junto con las reglas de formateo del archivo final, el código se llena de condicionales if/else, manipulación de strings manuales y métodos distintos según el formato como podrían serlo `exportar_csv()`, `exportar_txt()`, entre otros.

Las fuerzas en conflicto:

1. **Complejidad del Algoritmo de Construcción:** Armar la cadena de texto de un lote requiere procesar datos de múltiples fuentes de forma secuencial: encabezado de la cooperativa (R-8), datos del productor (R-1), sumatoria exacta de pesos en Decimal (D3.1) y metadatos de auditoría (R-10).

2. **Independencia del Formato Final:** La lógica que recorre los datos del lote para estructurarlos no debería saber si el resultado final será un archivo de ancho fijo o un archivo CSV. Por lo que debería funcionar de la misma manera independientemente cual sea la extensión del archivo de salida.

El síntoma en código:

```
# SÍNTOMA FEO: El objeto o servicio asume la responsabilidad de formatear textualmente.
# Si el formato cambia o entra uno nuevo, hay que venir a meter condicionales aquí.
from decimal import Decimal

class Lote:
    def __init__(self, id_lote: str, cooperativa: str, peso_kilos: Decimal, humedad: Decimal):
        self.id_lote = id_lote
        self.cooperativa = cooperativa
        self.peso_kilos = peso_kilos
        self.humedad = humedad

    def exportar_datos(self, formato: str):
        # Condicionales por nombre que arruinan la extensibilidad
        if formato == "POSICIONES_FIJAS_1998":
            # El R-6 exige posiciones fijas estrictas
            id_fijo = self.id_lote.ljust(10)[:10]
            coop_fija = self.cooperativa.ljust(15)[:15]
            peso_fijo = str(self.peso_kilos).zfill(8)[:8]
            return f"{id_fijo}{coop_fija}{peso_fijo}"

            elif formato == "CSV": #Logica del CSV
                return f"{self.id_lote},{self.cooperativa},{self.peso_kilos},{self.humedad}"

        # Si el auditor pide JSON, hay que meter otro 'elif' con la logica del JSON modificanfo el nucleo
        return ""
```

El objeto maneja la lógica del formato del texto dentro del reporte, lo que implica que si el formato cambia, o entra uno nuevo, se debe manipular el núcleo de este objeto y su lógica por cada cambio que se necesite hacer, lo cual es una mala práctica ya que roba tiempo al hacer cambios que se pueden evitar aplicando un buen patrón de diseño para esta sección del código.

3. La solución

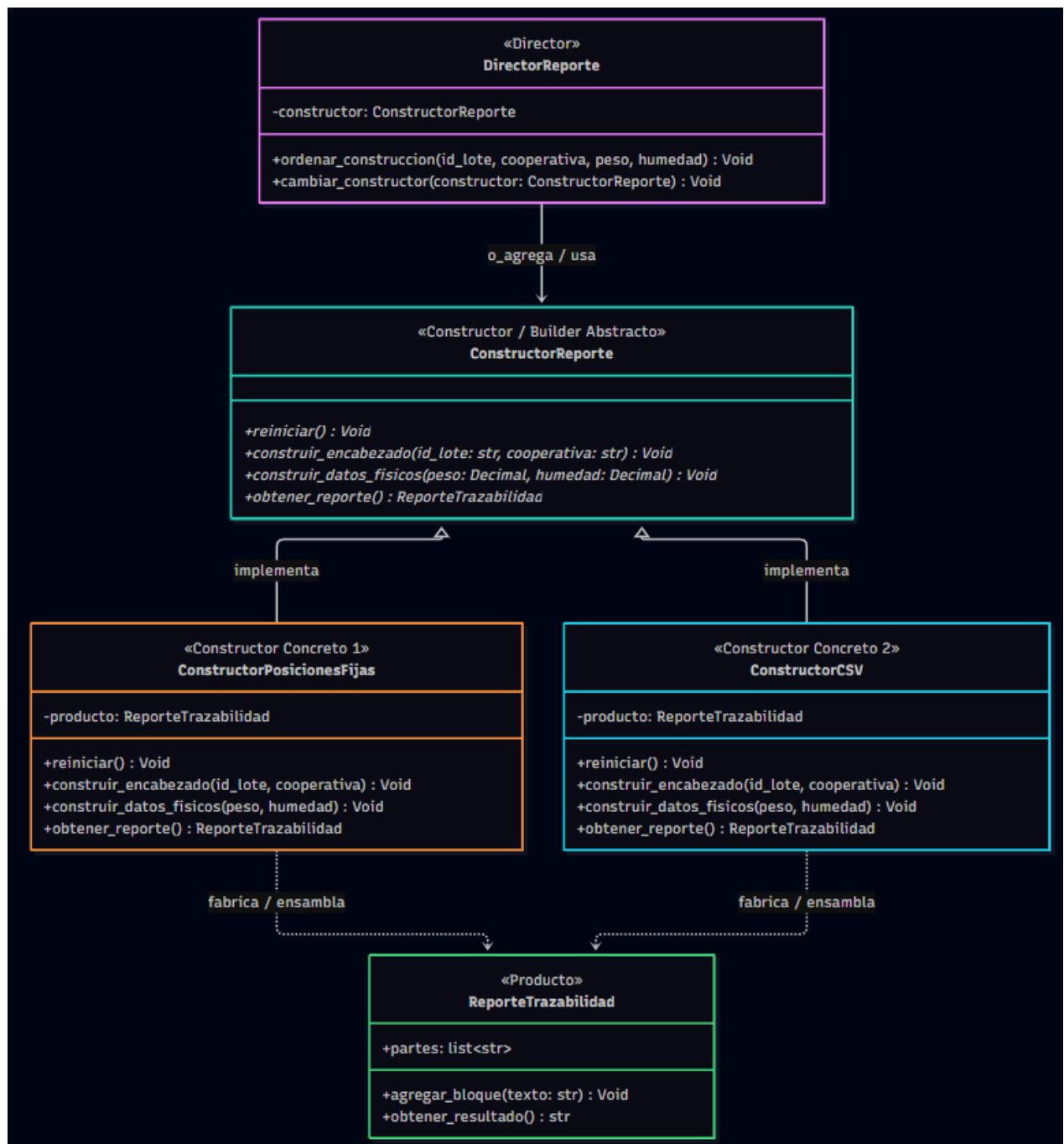
La solución al problema visto en el punto anterior se representa en el siguiente diagrama, cuyas secciones representan las siguientes cosas:

Director (DirectorReporte): Es el que conoce el orden secuencial de los pasos para armar el informe del ICAFE o las auditorías. No sabe nada de formatos de archivos de texto, solo llama a los métodos en el orden correcto.

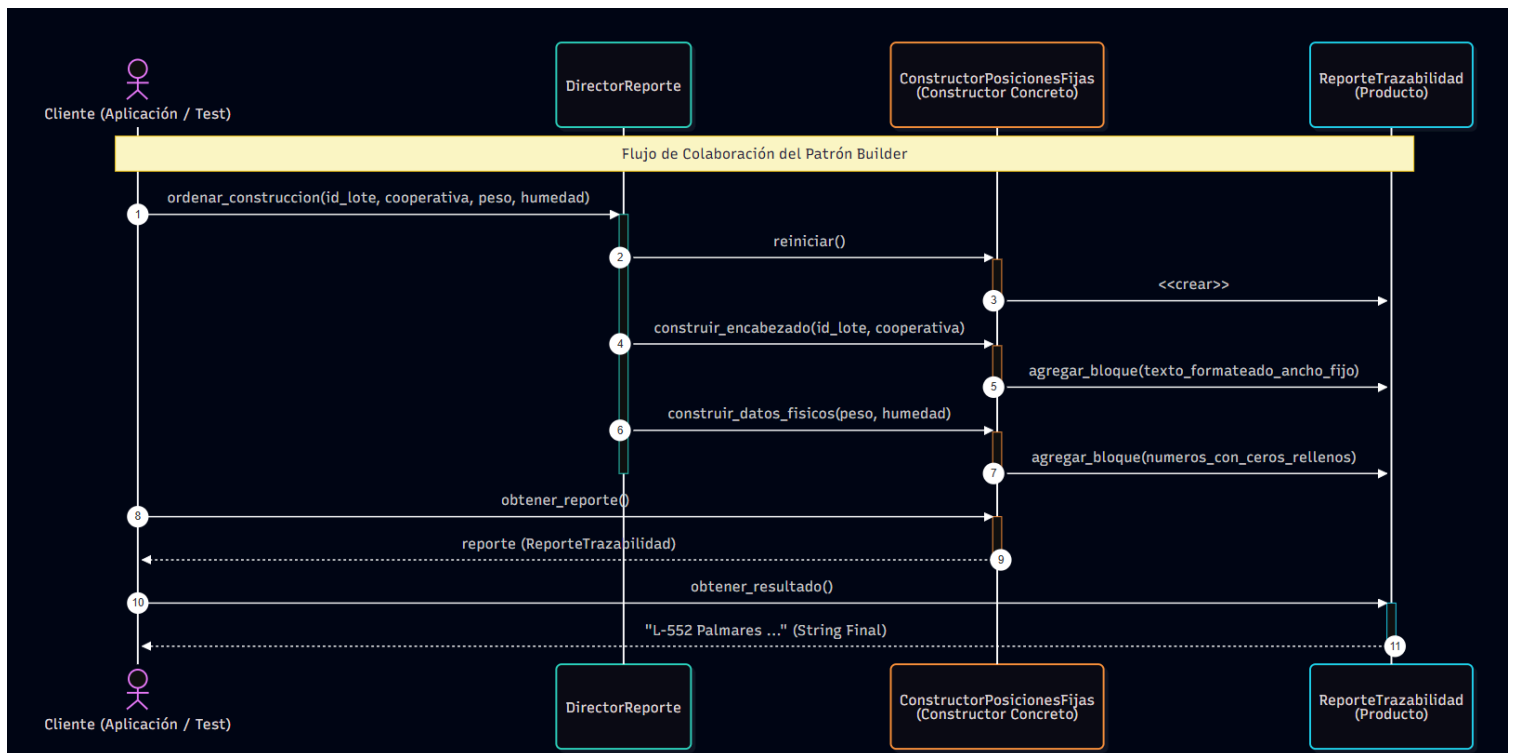
Builder Abstracto (ConstructorReporte): Clase abstracta que define los pasos que todos los constructores concretos deben implementar.

Builders Concretos (`ConstructorPosicionesFijas` y `ConstructorCSV`): Los encargados de la infraestructura que conocen cómo rellenar espacios en blanco o separar con comas como en el caso del CSV.

Producto (`ReporteTrazabilidad`): Objeto que acumula los bloques construidos y permite obtener el contenido textual del reporte terminado.



4. La colaboración



El diagrama anterior muestra la secuencia de la operación central, o en otras palabras, quién llama a quién cuando el patrón entra en acción. El flujo de este diagrama cuenta con 11 pasos los cuales cada uno representa una etapa crucial del proceso.

Pasos del 1 al 2: El cliente inicia el proceso delegando los datos crudos al `Director`. El `Director` ordena inmediatamente al `Builder` limpiar cualquier residuo e iniciar una instancia fresca del `Producto`.

Pasos del 3 al 6: El `Director` actúa como un capataz: dicta *qué* bloque se construye a continuación, pero el `Builder Concreto` es el único que sabe *cómo* formatear e inyectar ese bloque específico dentro del objeto `Producto`. El `Director` jamás toca ni conoce la representación interna del informe.

Pasos del 7 al 11: El `Director` termina su secuencia abstracta. El `Cliente` extrae directamente el `Producto` finalizado desde el `Builder` (`obtener_reporte()`), manteniendo al `Director` completamente desacoplado del tipo o formato de retorno.

5. El contrato

Precondición: `DirectorReporte` tiene asignado un objeto que cumple el contrato de `ConstructorReporte` y recibe `identificador`, `cooperativa`, `peso` y `humedad` válidos.

Poscondición: `ordenar_construccion()` deja en el constructor activo un `ReporteTrazabilidad` con el encabezado y los datos físicos, agregados en ese orden y con la representación elegida.

Invariante: El Director mantiene la misma secuencia de construcción y depende únicamente de `ConstructorReporte`, sin preguntar por el tipo concreto del constructor.

6. Implementación mínima

El código de la implementación en Python demuestra el patrón Builder mediante un `DirectorReporte` que coordina secuencialmente la construcción de reportes de lotes de café para exportación. Las subclases `ConstructorPosicionesFijas` y `ConstructorCSV` gestionan el formateo específico del objeto `ReporteTrazabilidad` sin alterar la lógica del director, lo que genera una construcción limpia y lista para recibir nuevos formatos sin necesidad de modificar el código ya hecho.

Puede revisar el código fuente en el enlace a nuestro repositorio de GitHub.

→ https://github.com/Andr3sCamp0s/Patron_Builder

7. Qué compra y qué paga

Este patrón compra independencia entre el proceso de construcción y la representación del producto y lo paga con más clases, estado temporal y coordinación. Aplica responsabilidad única porque el Director ordena los pasos y cada Builder se ocupa de un formato. Favorece abierto/cerrado e inversión de dependencias: se agrega otro formato implementando el contrato, sin cambiar el

Director. El costo es mantener la interfaz, los constructores concretos y el reinicio correcto del producto, aun cuando una función directa podría ser más corta.

8. Cuándo NO usarlo

- Caso 1: si CaféTrace solo tuviera que producir para siempre el archivo fijo del ICAFE y el reporte se formase con una única operación estable, una función `generar_archivo_icafe(lote)` sería más clara que un Director, una abstracción y varios constructores.
- Caso 2: no conviene usarlo para crear Peso o Entrega en “cafetrace/dominio/modelos.py.” Son objetos pequeños e inmutables cuyos datos obligatorios se conocen al mismo tiempo; un Builder introduciría estados parciales inválidos sin aportar variaciones reales de construcción.

9. En su Proyecto

Como el proyecto aún no ha sido implementado, los siguientes lugares son una propuesta de diseño para CaféTrace.

En una futura clase o `archivo exportador_icafe.py`, encargado de construir reportes de trazabilidad en distintos formatos. Aplicaría Builder porque el proceso de ensamblaje sería el mismo, pero la representación final podría ser de posiciones fijas, CSV o JSON.

No lo aplicaría en `modelos.py` para crear objetos pequeños como Peso, Lote o Entrega, porque tienen pocos atributos y pueden construirse directamente sin agregar directores y constructores.

10. El patrón vecino

El patrón de la mesa que más se parece a Builder es Abstract Factory porque ambos son creacionales y esconden clases concretas detrás de abstracciones. Sin embargo, no resuelven el mismo problema. Abstract Factory se usa para crear familias de productos relacionados y compatibles, por ejemplo una familia completa de controles para Windows y otra para Linux. Normalmente el cliente solicita cada

producto mediante operaciones independientes de la fábrica, como `crear_boton()` o `crear_ventana()`. Su decisión principal es cuál familia crear.

Builder, en cambio, se concentra en cómo ensamblar paso a paso un producto complejo. El Director conserva una secuencia de construcción y distintos Builders interpretan esa misma secuencia para producir representaciones diferentes. En CaféTrace no se elige una familia de objetos relacionados: se construye un reporte por etapas y se decide si esas etapas terminan representadas como posiciones fijas, CSV u otro formato. Incluso si ambos diagramas muestran una interfaz y varias implementaciones, la diferencia real está en la intención y en la colaboración: Abstract Factory selecciona familias; Builder separa la secuencia de ensamblaje de la representación final.